Lab 4 — Frogger

Conversation History

Software Engineering Lab — Tasks 1 to 4

Task 1	Vehicle collision detection
Task 2	Lives and respawn
Task 3	Goal completion and score tracking
Task 4	30-second countdown and timeout handling
Workflow	Testing, debugging, and separate Git commits

This document follows the work in order. It includes the user's main prompts, the implementation decisions, the variables and logic added, the issues noticed during testing, and the Git workflow. The focus is on explaining what was done for each task rather than reproducing every full code file.

1. Assignment context and submission plan

The project was a Pygame Frogger starter game. The starter already had a playable frog, moving vehicles, start and goal zones, rendering, a game loop, and an R key for restarting. The README identified a collision-detection bug and listed four tasks.

• **Task 1:** Fix vehicle collision detection.

• **Task 2:** Add three lives and respawn after collisions.

• **Task 3:** Detect goal completion, track score, and provide appropriate success feedback.

• **Task 4:** Add a 30-second timer for each attempt and handle timeouts.

The submission also required a short video of the original bug, a short video showing the completed game, and a link to the conversation history.

Repository and commit plan

The work was being prepared in the college submission repository under `~/college/Lab_04/AM123/01_frogger`, with the videos and PDF planned under `~/college/Lab_04/AM123`. The user planned to make separate commits for the before video, each task, and the final submission files, and to push later rather than push directly to the original SETAPESU26 repository.

Two commit details were mentioned during the work: `c3ce5a5 Lab_04:add before-change video of original game` and `d8a3183 Task 1: Fix vehicle collision detection`.

2. Task 1 — Fix vehicle collision detection

User's observation

> **User:** The smaller cars let the frog overlap visually before the collision was detected, while the wider trucks seemed to trigger collision sooner.

The original function converted a vehicle's x-coordinate into a grid column and compared that column with the frog's column. This did not check the actual overlap between the frog and vehicle shapes. Since cars were 40 pixels wide and trucks were 70 pixels wide, checking only the left-edge grid column could produce inconsistent-looking collision behavior.

What we changed

We changed `game/collisions.py` to use the actual rectangles of the frog and each vehicle. The function gets the frog's rectangle using `frog.get_rect(CELL_SIZE)`, gets each vehicle's rectangle using `vehicle.get_rect(CELL_SIZE)`, and checks overlap with Pygame's `colliderect()` method.

```
frog_rect = frog.get_rect(CELL_SIZE)

for vehicle in vehicles:
    vehicle_rect = vehicle.get_rect(CELL_SIZE)
    if frog_rect.colliderect(vehicle_rect):
        return True

return False
```

This made collision detection depend on actual rectangle overlap instead of a grid-column approximation. The user tested the change and confirmed that collision behavior worked. Task 1 was then committed separately.

3. Task 2 — Lives and respawn

What we needed

The game needed to start with three lives. Every vehicle collision should cost one life. If the player still has lives remaining, the frog should return to its starting position. When all lives are lost, the game should show Game Over and stop accepting movement until restart.

What we changed

In `GameEngine`, we added or used the state variables `self.lives = 3` and `self.game_over = False`. Collision handling decrements the lives count. If the count reaches zero, `self.game_over` becomes True; otherwise, `self.frog.reset()` returns the frog to the starting position.

The R key was handled before the Game Over movement guard so that restart remains possible after losing. Restarting restores three lives, clears Game Over, and rebuilds the frog and vehicles.

Debugging

> **User:** The frog was staying at the initial position.

This was traced to an indentation mistake in `game_engine.py`. After the indentation was corrected, the user reported that the behavior was working. A separate Task 2 commit was planned, but the conversation did not confirm its final commit hash.

4. Task 3 — Goal completion and score tracking

Understanding the requirement

> **User:** Task 3 says: “Detect successful goal completion and implement score tracking with an appropriate win state.”

The first implementation treated winning as a permanent state and stopped the game. The user pointed out that this would reset or end the gameplay after each goal, whereas the intended behavior was to keep playing and accumulate score during the same three-life session.

> **User:** I think we need this: currently, after each win the score resets to 0 again, but we want the score to continue during a complete three-life session. Only update the score back to 0 when the three lives are over.

We therefore treated each successful goal as a scoring event, rather than a permanent end to the game. Reaching the goal increments the score and resets the frog to its starting position. The score remains accumulated after goals and ordinary collisions.

Displaying goal success

> **User:** The goal-reached text should not persist for more than one second.

We kept a `self.won` flag, but used it only to control the temporary success message. We added `self.goal_message_time` to record when the goal was reached. When the goal is reached, the score increases, `self.won` becomes True, the current Pygame time is saved, and the frog resets.

During each update, the elapsed time is calculated with `pygame.time.get_ticks()`. When at least 1,000 milliseconds have passed, `self.won` becomes False and the message is no longer drawn.

```
if self.won:
    elapsed = pygame.time.get_ticks() - self.goal_message_time
    if elapsed >= 1000:
        self.won = False

if self.frog.row == GOAL_ROW:
    self.score += 1
    self.won = True
    self.goal_message_time = pygame.time.get_ticks()
    self.frog.reset()
```

The draw method displays `GOAL REACHED! Score: N` while `self.won` is True. We deliberately did not use `self.won` in the early-return checks for gameplay, because that would freeze the game after the first goal.

Score and restart behavior

The score stays visible through the session and remains visible during Game Over so the player can see the final result. Pressing R starts a new session and resets the score to zero. The score does not reset each time the frog reaches the goal.

Commit

After testing the goal message and score behavior, the recommended Task 3 commit was:

```
cd ~/college
git status
git add Lab_04/AM123/01_frogger/game/game_engine.py
git commit -m "Task 3: Implement goal and score tracking"
```

> **User:** I removed the old game_engine.py file, added the new one, and committed it. Does that create a conflict with the previous commit?

The explanation was that Git records the new file state in the next commit. Earlier commits remain in history. Replacing a file does not itself create a conflict; conflicts usually occur when Git merges overlapping changes.

5. Task 4 — Countdown timer and timeout handling

Requirement

Each attempt must have a 30-second time limit. If the timer reaches zero, the player loses one life. If lives remain, the frog respawns and a new attempt starts; if no lives remain, Game Over appears.

What we added

We added `ATTEMPT_TIME = 30` and `self.attempt_start_time`. The start time is set when the game begins, when the player restarts, and whenever a new attempt begins after a collision, timeout, or successful goal.

The timer uses `pygame.time.get_ticks()`, which measures elapsed milliseconds. We calculate elapsed seconds from the difference between the current time and the attempt start time. When elapsed time reaches 30 seconds, the same life-loss logic used for collisions is called.

```
ATTEMPT_TIME = 30

current_time = pygame.time.get_ticks()
elapsed_seconds = (
    current_time - self.attempt_start_time
) // 1000

if elapsed_seconds >= ATTEMPT_TIME:
    self.lose_life()
    return
```

One shared life-loss method

We added `lose_life()` so timeout and collision follow the same rules. It subtracts one life and hides the goal message. If no lives remain, it sets Game Over; otherwise, it resets the frog and starts a fresh timer.

```
def lose_life(self):
    self.lives -= 1
    self.won = False

    if self.lives <= 0:
        self.game_over = True
    else:
        self.frog.reset()
        self.attempt_start_time = pygame.time.get_ticks()
```

Showing the remaining time

The draw method calculates how many seconds have elapsed, subtracts that from 30, and displays the remaining time. The remaining value is clamped to zero so the display never shows a negative countdown.

```
elapsed_seconds = (
    pygame.time.get_ticks() - self.attempt_start_time
) // 1000

remaining = max(0, ATTEMPT_TIME - elapsed_seconds)

renderer.draw_text(
    surface, font, f"Time: {remaining}", (10, 58)
)
```

> **User:** Yeah, it's working properly, but now I need to wait 30 seconds to show the behavior for the final video!

To record the timeout more quickly, we discussed temporarily using a five-second timer in testing mode, while preserving 30 seconds for the final submission.

```
TESTING_MODE = True
ATTEMPT_TIME = 5 if TESTING_MODE else 30
```

For the final submission, testing mode must be False so that the actual timer is 30 seconds:

```
TESTING_MODE = False
ATTEMPT_TIME = 5 if TESTING_MODE else 30
```

The user reported that the completed timer behavior worked properly. The final code must be checked to ensure the testing setting is disabled before submission.

6. Final behavior across all tasks

• **Startup:** Three lives, score 0, and a 30-second timer.

• **Collision:** One life is lost. The frog respawns and the timer restarts if lives remain.

• **Goal reached:** Score increases by one, the goal message appears for about one second, and the frog returns to the starting position.

• **Continued play:** The score accumulates across successful goals and does not reset after an ordinary collision.

• **Timeout:** The attempt ends when 30 seconds expire, costing one life.

• **Game Over:** After all three lives are lost, gameplay stops and the Game Over banner appears.

• **Restart:** Pressing R restores three lives, resets the score to zero, clears the message and Game Over states, rebuilds entities, and starts a new timer.

7. Git workflow and submission

Separate commits

The user wanted to keep the work organized into separate commits rather than combine all tasks into one. The planned order was the before-change video, Task 1, Task 2, Task 3, Task 4, and then the final video/PDF and remaining submission files. The user planned to push after completing and checking the work.

```
git log --oneline -5
git status
```

These commands were recommended to check the commit history and see whether there were uncommitted changes.

Planned folder structure

```
~/college/Lab_04/AM123/
    01_frogger/
        README.md
        requirements.txt
        main.py
        game/
            __init__.py
            game_engine.py
            frog.py
            vehicle.py
            collisions.py
            renderer.py
    before.mp4
    after.mp4
    chat_history.pdf
```

This reflects the intended submission organization. The actual folder contents and final commit should be verified locally.

8. Final checklist

• Run the game and confirm it starts without errors.

• Check that collisions use actual rectangle overlap.

• Check three lives, respawn, and Game Over.

• Check that every goal increases the score and the message disappears after one second.

• Check that the score persists across goals and lives within the session.

• Ensure the submitted timer is 30 seconds, not the temporary five-second testing value.

• Check that timeout costs one life and respawns the frog if lives remain.

• Keep the before video showing the original bug and record the after video showing the completed features.

- Check git status and git log before the final commit.
- Ensure the files are in the college submission repository and push only when ready.